BudgetMesh: what we compare against, and why

All numbers are measured runs. 13 Nov 2026.

1 The main comparison

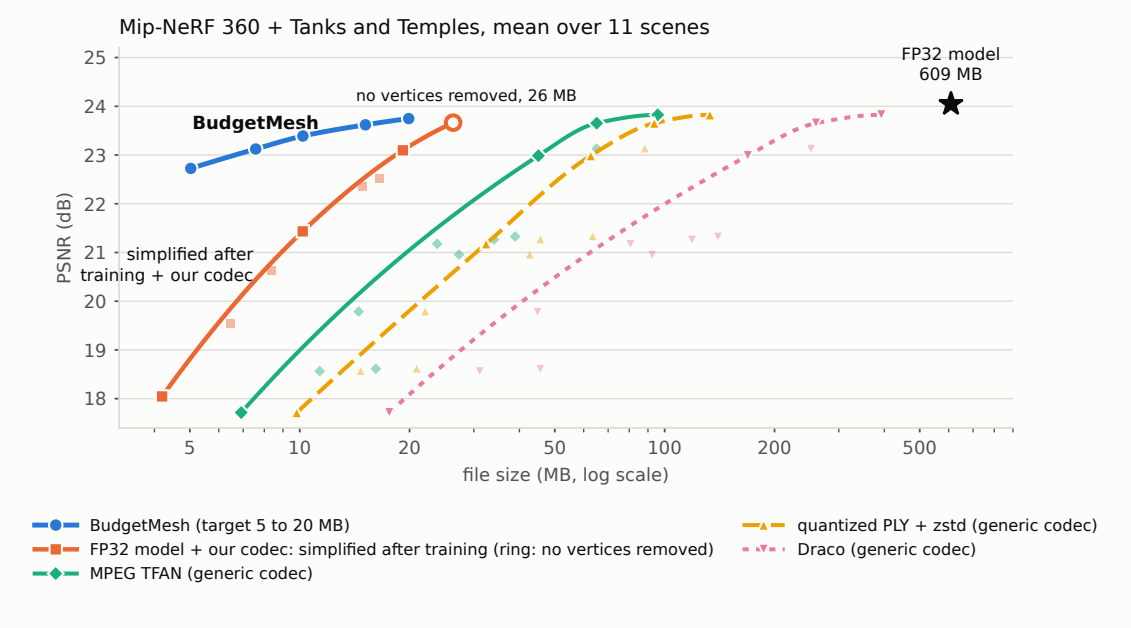


Figure 1: Quality (PSNR) against file size. Mean of 11 captured scenes (Mip-NeRF 360, Tanks and Temples). Each curve keeps the best points of its method.

What each curve is.

- **FP32 model (star).** MeshSplatting as released, default settings, saved with no compression. 609 MB.
- **Draco (pink).** Google's mesh compressor, used as it is. We give it the MeshSplatting mesh at full size and at three smaller sizes (vertices removed after training, then fine-tuned). Colours are stored with 8, 6 or 4 bits.
- **MPEG TFAN (green).** The MPEG mesh compressor, used as it is. Same meshes as Draco.
- **Quantized PLY + zstd (yellow).** Round the numbers, write them to a PLY file, compress the file with zstd, a general-purpose compressor. Same meshes as Draco.
- **FP32 model + our codec (orange).** The same kind of mesh again: the MeshSplatting mesh, at full size (the ring) and with vertices removed after training. Only the compressor is different: ours.
- **BudgetMesh (blue).** Our full system. The mesh is brought to the target size while it is still training, then compressed with our codec.

What the figure shows.

1. **As a codec, ours compresses better.** Orange against green, yellow and pink: the meshes are the same, only the compressor changes. For the full-size mesh at about 23.65 dB, our file is 26 MB, TFAN's is 65 MB, zstd's is 94 MB and Draco's is 260 MB. Over the whole curve we are 2.4 to 2.5 times smaller than TFAN, the best of the three.
2. **The size has to be controlled during training.** Blue against orange: the compressor is the same, the only difference is when the mesh is brought to size. At 5, 10 and 20 MB, sizing during training is 4.0, 2.0 and 0.6 dB better than removing vertices after training.
3. **Both together.** For the same PSNR, TFAN needs 4.1 to 8.2 times more bytes than BudgetMesh. This number contains both gains, the codec and the sizing during training.

2 Why we do not build more baselines of our own

- Any baseline needs a trained scene. A trained scene comes from the original MeshSplatting. The only handle on size there is the vertex cap.
- In the released code the cap is one fixed number, 4 million vertices, the same for every scene. It is a ceiling, not a way to choose a file size.
- We are the ones who started tuning it, because it is the most direct handle on the number of vertices.
- It does not behave like a size setting (Figure 2). Without trying several caps on a scene, nobody can know which cap gives which size, and a cap found on one scene does not carry over to another.
- Taking the cap from other scenes of the same dataset (a prior) is not a fair comparison either. Our method is given only

the scene and the target size.

- So the vertex cap is not a baseline. It is one row in the ablation and it answers one question: *how much better is the controller than tuning the cap by hand?*

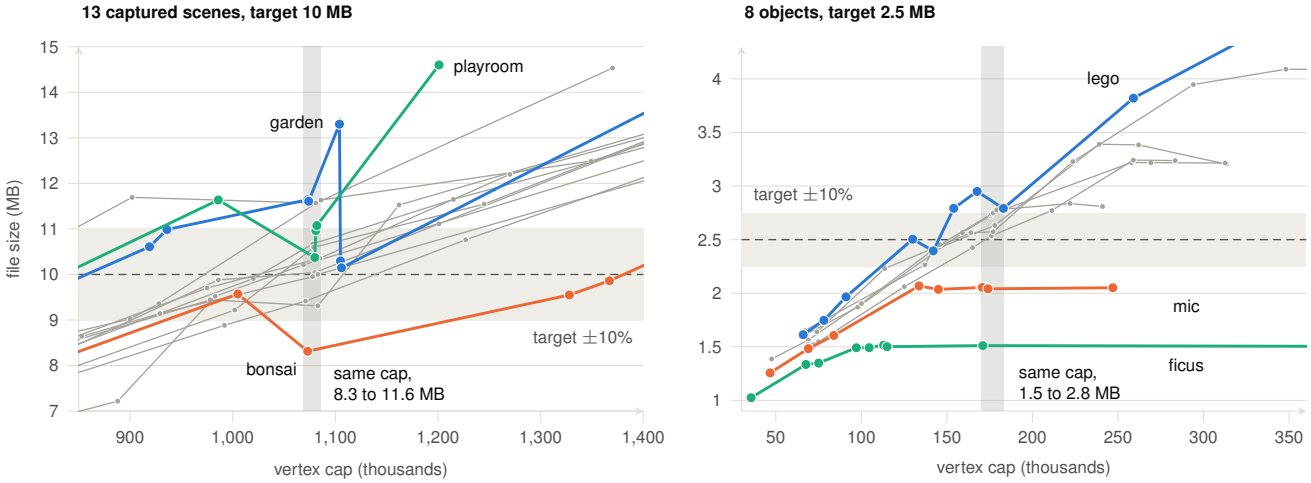


Figure 2: File size (with our codec) against vertex cap, one line per scene, every cap run in the plotted range. Three scenes per panel are coloured, the others are grey. Dashed line and light band: target and ±10%. Dark vertical strip: the cap at which every scene of the panel was trained.

- **Same cap, different scenes, different sizes.** With a cap of about 1.08 million the 13 captured scenes give files from 8.3 to 11.6 MB. With a cap of about 176 thousand the 8 objects give files from 1.5 to 2.8 MB.
- **Same scene, almost the same cap, different sizes.** Garden: a cap of 1,104,000 gives 13.3 MB, a cap of 1,105,000 gives 10.3 MB.
- **A larger cap can give a smaller file.** Bonsai: 1,005,000 gives 9.6 MB, 1,073,000 gives 8.3 MB.
- **Raising the cap can do nothing.** Mic stays at about 2.05 MB from a cap of 134 thousand up to 247 thousand. Ficus stays at 1.5 MB from 97 thousand up to 515 thousand.
- **Over all cap runs.** Take two runs of the same scene whose caps are less than 1% apart: 11 such pairs, and in 4 of them the larger cap gave the smaller file. Less than 5% apart: 29 pairs, 12 with the larger cap giving the smaller file, 8 with the file changing by more than 10%.

3 The ablation: controller against tuning the cap by hand

How we count.

- Both sides get the same input: the scene and the target size. Nothing from other scenes.
- Without a prior, the cap user must first run the released default (cap 4 million) once to see how large the scene is. We call this run 0, and its time is counted.
- Then come the capped trainings, until the file is within ±10% of the target, at most three.
- Time for the cap = run 0 + all capped trainings. We report the slowest scene (worst case). Encoding time is reported separately for both sides.

What is measured so far. 21 scenes, one target each: 10 MB for the 13 captured scenes, 2.5 MB for the 8 objects. In these runs the cap was still taken from other scenes of the same dataset, so the cap column had help that the rules above do not allow.

	Vertex cap, tuned by hand	Controller (ours)
Within ±10% after 1 training	15 / 21	20 / 21
Within ±10% after 3 trainings	18 / 21	20 / 21 (never retrains)
Within ±2% after 1 training	3 / 21	16 / 21
Never within ±10%	drjohnson, mic, ficus	ficus
PSNR at the same file size	reference	+0.33 dB (better in 34 of 36)
Slowest scene that reached ±10%	81 min, 2 trainings, run 0 not yet added	71 min, 1 training

Still to add: [time of run 0 for each scene, added to the cap column] and [encoding time for both columns].